

Brute Force

A straightforward approach, usually based directly on the problem's statement and definitions of the concepts involved

Examples:

1. Computing a^n ($a > 0$, n a nonnegative integer)
2. Computing $n!$
3. Multiplying two matrices
4. Searching for a key of a given value in a list

Brute-Force Sorting Algorithm

Selection Sort Scan the array to find its smallest element and swap it with the first element. Then, starting with the second element, scan the elements to the right of it to find the smallest among them and swap it with the second elements. Generally, on pass i ($0 \leq i \leq n-2$), find the smallest element in $A[i..n-1]$ and swap it with $A[i]$:

$A[0] \leq \dots \leq A[i-1] \mid A[i], \dots, A[\min], \dots, A[n-1]$
in their final positions

Example: 7 3 2 5



Selection Sort

	89	45	68	90	29	34	17
17		45	68	90	29	34	89
17	29		68	90	45	34	89
17	29	34		90	45	68	89
17	29	34	45		90	68	89
17	29	34	45	68		90	89
17	29	34	45	68	89		90

FIGURE 3.1 Example of sorting with selection sort. Each line corresponds to one iteration of the algorithm, i.e., a pass through the list tail to the right of the vertical bar; an element in bold indicates the smallest element found. Elements to the left of the vertical bar are in their final positions and are not considered in this and subsequent iterations.

Analysis of Selection Sort

ALGORITHM *SelectionSort*($A[0..n - 1]$)

//Sorts a given array by selection sort

//Input: An array $A[0..n - 1]$ of orderable elements

//Output: Array $A[0..n - 1]$ sorted in ascending order

for $i \leftarrow 0$ **to** $n - 2$ **do**

$min \leftarrow i$

for $j \leftarrow i + 1$ **to** $n - 1$ **do**

if $A[j] < A[min]$ $min \leftarrow j$

 swap $A[i]$ and $A[min]$

Time efficiency:

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = ?$$

Space efficiency:

Stability:

Bubble Sort

ALGORITHM *BubbleSort*($A[0..n - 1]$)

//Sorts a given array by bubble sort

//Input: An array $A[0..n - 1]$ of orderable elements

//Output: Array $A[0..n - 1]$ sorted in ascending order

for $i \leftarrow 0$ **to** $n - 2$ **do**

for $j \leftarrow 0$ **to** $n - 2 - i$ **do**

if $A[j + 1] < A[j]$ swap $A[j]$ and $A[j + 1]$

Time efficiency:

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=0}^{n-2-i} 1 = \Theta(n^2) ?$$

$\text{Swap}_{\text{worst}}(n) = C(n) ?$

89	$\overset{?}{\leftrightarrow}$	45		68		90		29		34		17
45		89	$\overset{?}{\leftrightarrow}$	68		90		29		34		17
45		68		89	$\overset{?}{\leftrightarrow}$	90	$\overset{?}{\leftrightarrow}$	29		34		17
45		68		89		29		90	$\overset{?}{\leftrightarrow}$	34		17
45		68		89		29		34		90	$\overset{?}{\leftrightarrow}$	17
45		68		89		29		34		17		90
45	$\overset{?}{\leftrightarrow}$	68	$\overset{?}{\leftrightarrow}$	89	$\overset{?}{\leftrightarrow}$	29		34		17		90
45		68		29		89	$\overset{?}{\leftrightarrow}$	34		17		90
45		68		29		34		89	$\overset{?}{\leftrightarrow}$	17		90
45		68		29		34		17		89		90

etc.

FIGURE 3.2 First two passes of bubble sort on the list 89, 45, 68, 90, 29, 34, 17. A new line is shown after a swap of two elements is done. The elements to the right of the vertical bar are in their final positions and are not considered in subsequent iterations of the algorithm.

Sequential Search

ALGORITHM *SequentialSearch*($A[0..n - 1]$, K)

//Searches for a given value in a given array by sequential search

//Input: An array $A[0..n - 1]$ and a search key K

//Output: The index of the first element of A that matches K

// or -1 if there are no matching elements

$i \leftarrow 0$

while $i < n$ **and** $A[i] \neq K$ **do**

$i \leftarrow i + 1$

if $i < n$ **return** i

else return -1

Sequential Search

ALGORITHM *SequentialSearch2*($A[0..n]$, K)

//Implements sequential search with a search key as a sentinel

//Input: An array A of n elements and a search key K

//Output: The index of the first element in $A[0..n - 1]$ whose value is

// equal to K or -1 if no such element is found

$A[n] \leftarrow K$

$i \leftarrow 0$

while $A[i] \neq K$ **do**

$i \leftarrow i + 1$

if $i < n$ **return** i

else return -1

Brute-Force String Matching

- pattern: a string of m characters to search for
- text: a (longer) string of n characters to search in
- problem: find a substring in the text that matches the pattern

Brute-force algorithm

Step 1 Align pattern at beginning of text

Step 2 Moving from left to right, compare each character of pattern to the corresponding character in text until

- all characters are found to match (successful search); or
- a mismatch is detected

Step 3 While pattern is not found and the text is not yet exhausted, realign pattern one position to the right and repeat Step 2

Examples of Brute-Force String Matching

Pattern: 001011

Text: 10010101101001100101111010

Pattern: happy

Text: It is never too late to have a happy childhood.

Pseudocode and Efficiency

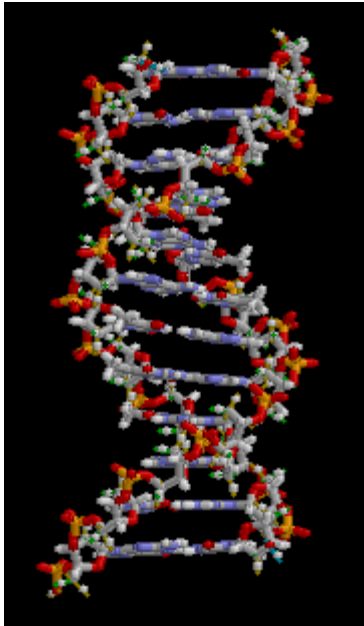
ALGORITHM *BruteForceStringMatch*($T[0..n - 1]$, $P[0..m - 1]$)

//Implements brute-force string matching
//Input: An array $T[0..n - 1]$ of n characters representing a text and
// an array $P[0..m - 1]$ of m characters representing a pattern
//Output: The index of the first character in the text that starts a
// matching substring or -1 if the search is unsuccessful

```
for  $i \leftarrow 0$  to  $n - m$  do  
     $j \leftarrow 0$   
    while  $j < m$  and  $P[j] = T[i + j]$  do  
         $j \leftarrow j + 1$   
    if  $j = m$  return  $i$   
return  $-1$ 
```

Efficiency:

DNA



Rules of the International Union of Pure and Applied Chemistry ([IUPAC](http://www.iupac-nmr.org)) defined:

A = adenine **C** = cytosine
G = guanine **T** = thymine
R = G A (purine) **Y** = T C (pyrimidine)
K = G T (keto) **M** = A C (amino)
S = G C (strong bonds) **W** = A T (weak bonds)
B = G T C (all but A) **D** = G A T (all but C)
H = A C T (all but G) **V** = G C A (all but T)
N = A G C T (any)

```

12854400 tcaaaagtaagttagataaaacatgatcattcacagggtcagatggttttaaaaaaaatcattatggtgtacatcacatgtagacaataacttcagaattcattc
12854200 tggactaccagaattgagttacctagtagtacttctcaattctattttaccctaactgtcctaataaatacaagtagtcttagcctcttcggttttatgatccctc
12854000 taggaaaaagttaattgttacggccaatcacttttttaacagcccaacaacatatattagctccaaatcattttttccctagaataattctcaacct
12853800 attgtccactcaaaacgtgacaaatggaggtctaaagggagaccatacttgactcatttttagagctaggatcagacagagtagatttttgccataactc
12853600 cttgtaaatgtattcacatttcatttcccaagaaaaatagactgatgaagaaatatcagatatgacaagggcgtgtcgttttaggttacgtaactctaca
12853400 aggttttaggggtctcaatataaacacacaaagcagatagaagaagcaaacattcacaaacagacaATGACATCTCTCCATACGTTACTCTTCTCTTCTCT
12853200 TCTTTTCTTCATCGTCTTTCCAACTTCACGTTTCTCTCCACTTATTGTTTCAGgttcgttcttagtttgcctctttacatacacagactctacacac
12853000 tcacttattgggtttctttcaattgtgaacagAGTTTCAATTGGGAGTCATGGAAAGAAAGAGGAGGATTCTACAATTCTCTCCACAACCTCCATTGACG
12852800 ACATAGCCAACGCTGGAATCACTCATCTTTGGCTTCTCTCTCTCTCTCAATCCGTTGCTCTCTGAAGgttccatttctgctttactctttacacattcaca
12852600 taccaatcttgttactcagcaatcttctattctctcagGTTACTTACCGGGAAAGCTATACGATCTAAACAGCTCCAAATACGGTTCCAGAGCGGAACTGA
12852400 AATCGTTAATCAAAGCGTTGAATCAAAAAGGAATAAAAGCTTTGGCTGATATAGTGATTAACACAGACAGCTGAGAGGAAAGACGATAAATGTGGATA
12852200 CTGTTATTTTCGAAGGTGGGACTTCCGATGATCGTCTTGATTGGGATCTCTCTTTGTCTGCCGCAATGACCCCTAAATTTCCCGGTACCGGAAACCTCGAC
12852000 ACCGGAGGAGATTTTGTGGAGCGCCGACATCGACCACCTTAAACCCTAGAGTTTCAGAAAGAGTTGTCCGAATGGATGAATTTGGCTTAAACTGAAATCG
12851800 GATTCCATGGTTGGAGATTGATTATGTTTCGAGGTTATGCATCTTCCATCACCAAATTTATCGTTCAgtaaatcacatatgaattctcaaatatcagac
12851600 aacagattatgtatataaagaacataggttgagataattatttactatttactatattagatataagttatcataggttgataggggttatttactactatttagt
12851400 ataagaaacataagtcgaatgcaatcaataaagaatatataaagaagttcactactgattatgtgataaattctctgttttttggatacacagAATACATC
12851200 ACCGGATTTTGGCGTGGGTGAGAAATGGGACCATATGAAGTACGGAGGAGACGGGAACTAGACTATGATCAGAACGAGCATCGGTCCGGGTCTCAAAACAG
12851000 TGGATCGAGGAAGCGGGTGGTGGTGTGTTGACAGCTTTTGATTTTCAACCAAAAGGATCTTACAGTCTGTCTGTCAAAGGTGAGCTTTGGAGACTAAAGG
12850800 ACTCGCAGGGAAAACCGCTTGGTATGATAGGAATCATGCCGGAAACGCTGTACATTTATAGATAAACCATGATACATTTAGAAAGTGGGTTTCCCTTC
12850600 TGATAAAGTCTTGCTTGGATACGTTTATATCTTACTTACTTACCCAGGAACCTCTTGCATTgttaagtatcatttttagtatgtagctatactatttacaactac
12850400 aatcttgtttagtatgttatttttgttgcagTTTATAATCATTACATAGGAGTGGGACTAAAAGAGAGCATCTCAAAGCTGGTGCTATTCAGGAACAAAA
12850200 ATGGGATTTGGTAGCACAAAGCTCTGTAACGATAAAAGCGGACAGAGCGGATCTCTACTTGGCTATGATTGATGATAAAGTTTATCATGAAGATTGGACCAA
12850000 GCAAGATGTGGGAACACTTGTCTCTTAATTTTGTCTTATTCAGGCTTGACTTTGCTGCTGGGAGAAGAAGTAAgcataactcgaatcata
12849800 agaaaaagtaatcgaatgtatcttcttcttctttaaataaaaacattttgcagctatcgaagatatgtataaataaaaatgataaagaataacctaaa
12849600 taaaaaagagcactagtgtgtttaaaggtatacaactccagtgaaagaaaagagtccaagtgaagaagtgccaactttgagaataagtatggaaagtttc
12849400 catcgtttgggttggatgatacaactaatatattatatttggccgactcgtataagatttggagcattcgaataattatgatgtcttaacca
12849200 cacaactactgccaaatcagaacgaattatattattgtagaagaagaaaaaaagattggtgggaagtggaacagtttagacaggttaatttcgaataaa
  
```

A
T
G
C
2
5
0
0
1

Homework

Word Puzzle (exercise 3.2 question 10)

ADERTYWIOPAPROJECTSZXCVGRM
UIOWISCONSINXCVLMQWADTRIO
GSISTERVRPBOMAWROCNEVELEDF
XNVCIRDECKMNETICVDYRDNUALZ
SGJLSMCWEATYARPTEDFGUHKTZM
GRANOISYDLRTHRULSABDSRZWSM
NYEUHJERVEFCMDOTSIWIIOHHTC
ILKHDADVTIMMBOADNGRNKKCIH
DIVITDEHERIEHNMZQEJHNVRTJB
DMHSDOAUPDLCXEKZHTVBCIDEWS
EARPANRFRASUKGZCHFBDNMTLMU
BFYGUTFBCEITSIYHQAWEATYRSP
TKHFDCVBMMQETRPSIERECORGAU
EDFSHJJODXCABLBOUQARWSHEQM
WJNUJMHKOLIDLOLIIBABIESBTG

ADRIEL	CALEB	DAD	FAMILY	JADON	NOISY	WET BEDDING
ADVENTURE	CEDRIC	DAMON	FUN	LAUNDRY	ONE GIRL	WISCONSIN
BABIES	CHERISH	DORAN	GREAT WHITE	LOVE	PROJECTS	WORK
BROTHERS	CHRISTIAN	ETHAN	GROCERIES	MARTIN	SISTER	
BUSY	CUTE	ELEVEN	HOMESCHOOLED	MOM	TWINS	

Brute-Force Polynomial Evaluation

Problem: Find the value of polynomial

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x^1 + a_0$$

at a point $x = x_0$

Brute-force algorithm

$p \leftarrow 0.0$

for $i \leftarrow n$ **downto** 0 **do**

$power \leftarrow 1$

for $j \leftarrow 1$ **to** i **do** *//compute x^i*

$power \leftarrow power * x$

$p \leftarrow p + a[i] * power$

return p

Efficiency:

Polynomial Evaluation: Improvement

We can do better by evaluating from right to left:

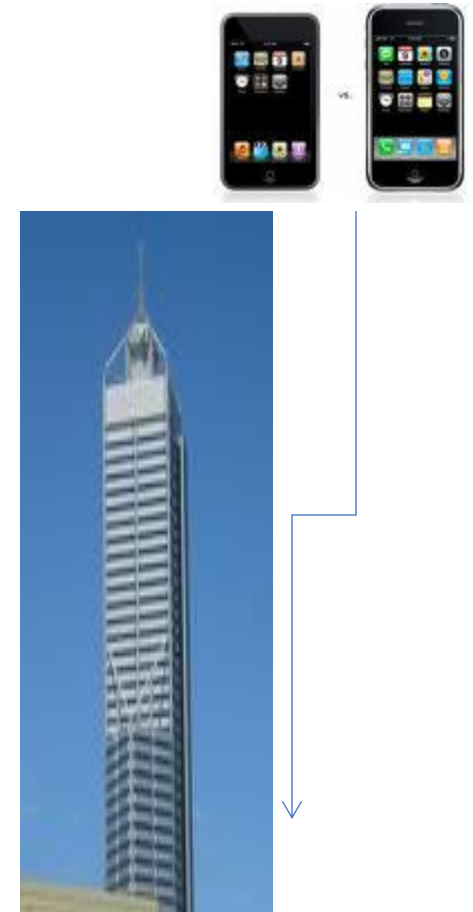
Better brute-force algorithm

```
 $p \leftarrow a[0]$   
 $power \leftarrow 1$   
for  $i \leftarrow 1$  to  $n$  do  
     $power \leftarrow power * x$   
     $p \leftarrow p + a[i] * power$   
return  $p$ 
```

Efficiency:

Gadget Testing

- A firm wants to determine the highest floor of its n -story headquarters from which a gadget can fall with no impact on the gadget's functionality. Two identical gadgets are given to experiment with. Design an algorithm in the best efficiency call.



Closest-Pair Problem

Find the two closest points in a set of n points (in the two-dimensional Cartesian plane).

Brute-force algorithm

Compute the distance between every pair of distinct points and return the indexes of the points for which the distance is the smallest.

Closest-Pair Brute-Force Algorithm (cont.)

ALGORITHM *BruteForceClosestPoints(P)*

//Input: A list P of n ($n \geq 2$) points $P_1 = (x_1, y_1), \dots, P_n = (x_n, y_n)$

//Output: Indices $index1$ and $index2$ of the closest pair of points

$dmin \leftarrow \infty$

for $i \leftarrow 1$ **to** $n - 1$ **do**

for $j \leftarrow i + 1$ **to** n **do**

$d \leftarrow \text{sqrt}((x_i - x_j)^2 + (y_i - y_j)^2)$ //sqrt is the square root function

if $d < dmin$

$dmin \leftarrow d; index1 \leftarrow i; index2 \leftarrow j$

return $index1, index2$

Efficiency:

$$\sum_{i=1}^{n-1} \sum_{j=i+1}^n 2 = 2 \sum_{i=1}^{n-1} (n-i) = 2[(n-1) + (n-2) + \dots + 1] = (n-1)n$$

$$\in \Theta(n^2)$$

How to make it faster?

Convex Hull Problem

- Definition: Convex

- A set of points (finite or infinite) in the plane is called convex if for any two points P and Q in the set, the entire line segment with the endpoints at P and Q belongs to the set.
- Convex means curving out

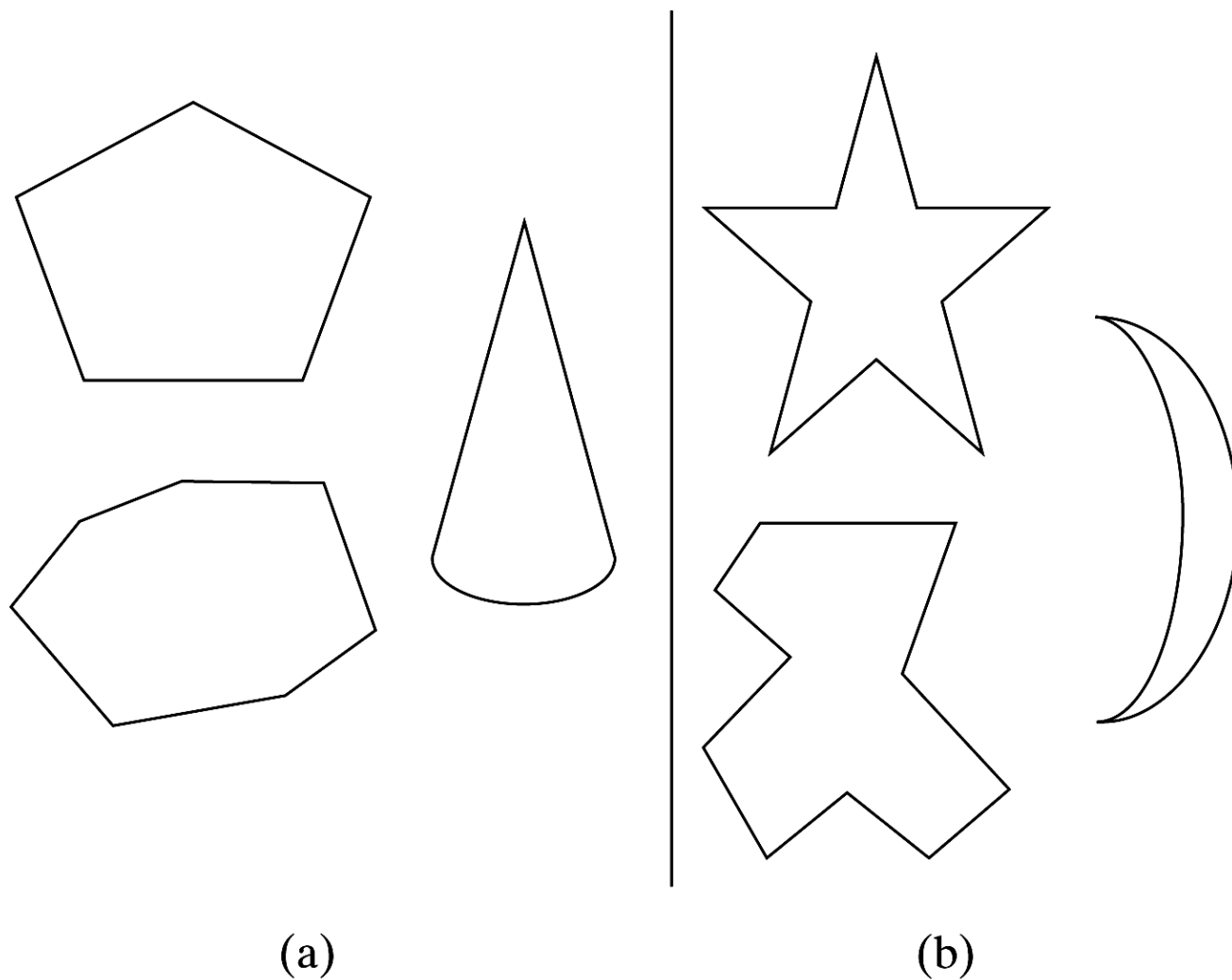


FIGURE 3.4 (a) Convex sets. (b) Sets that are not convex.

Convex Hull Problem

- Definition: Convex Hull
 - Convex hull of a set of n points in the plane is the smallest convex polygon that contains all of them (inside or on the boundary.)
 - Hull means body

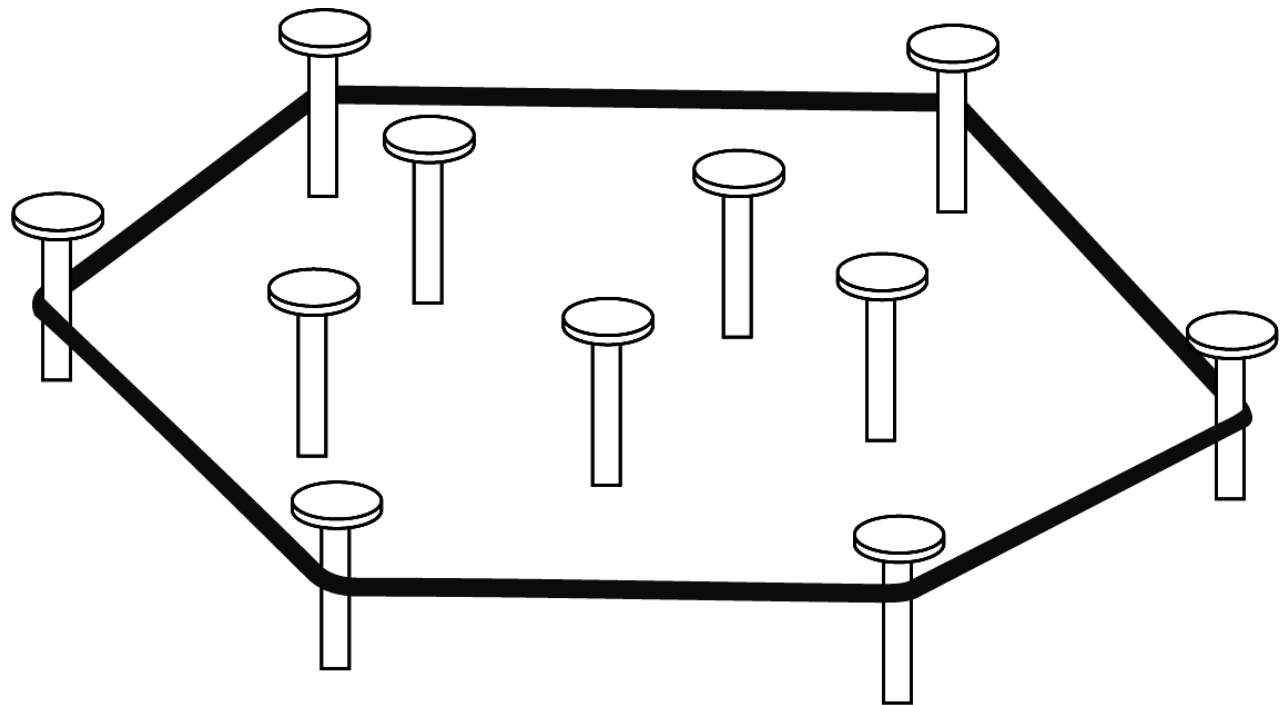


FIGURE 3.5 Rubber-band interpretation of the convex hull

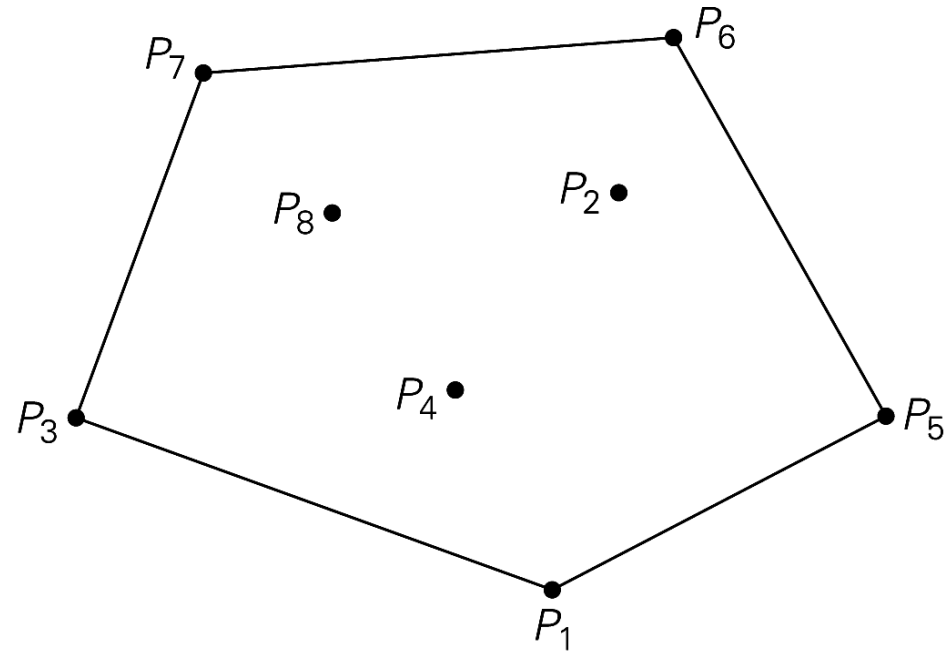


FIGURE 3.6 The convex hull for this set of eight points is the convex polygon with vertices at P_1 , P_5 , P_6 , P_7 , and P_3 .

Convex Hull Homework

- Linear equation of two points (x_1, y_1) and (x_2, y_2)

is $ax + by = c$ where

$$a = y_2 - y_1 \quad b = x_1 - x_2 \quad c = x_1 y_2 - x_2 y_1$$

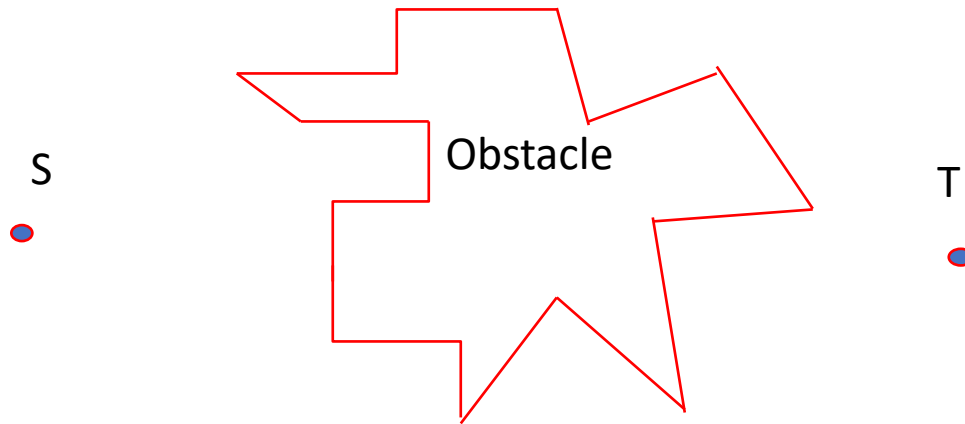
- $ax_k + by_k = c$ means (x_k, y_k) is on the line.
- On two half-planes divided by the line,
 - all points (x_k, y_k) that $Ax_k + by_k < c$ is on one plane.
 - all points (x_k, y_k) that $Ax_k + by_k > c$ is on another plane.

Convex Hull Homework

- State Convex Hull problem.
- Give an example of convex hull in x-y plane.
- Write a psedo code to solve convex hull problem using $ax + by = c$

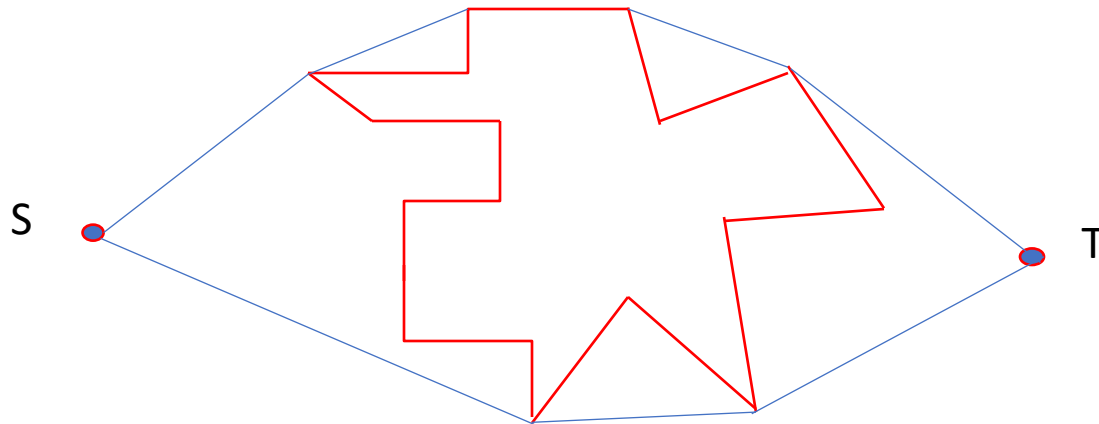
Convex Hull Applications: Motion Planning

- **Robot motion planning.** Find the shortest path in the plane from s to t that avoids a polygonal obstacle.



Convex Hull Applications

- Robot motion planning.



Fact: The shortest path is either straight line from s to t or it is one of two polygonal chains of convex hull.

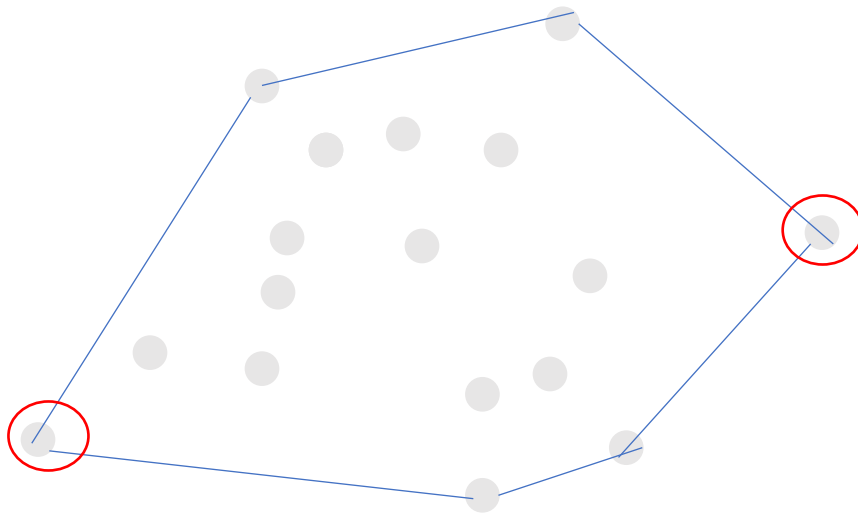
Convex Hull Application: Farthest Pair

- **Farthest pair problem:** Given N points in the plane, find a pair of points with the largest Euclidean distance between them



Convex Hull Application: Farthest Pair

- Farthest pair problem



Fact: Farthest pair of points are extreme points on convex hull.

Brute-Force Strengths and Weaknesses

- Strengths

- wide applicability
- simplicity
- yields reasonable algorithms for some important problems (e.g., matrix multiplication, sorting, searching, string matching)

- Weaknesses

- rarely yields efficient algorithms
- some brute-force algorithms are unacceptably slow
- not as constructive as some other design techniques

Exhaustive Search

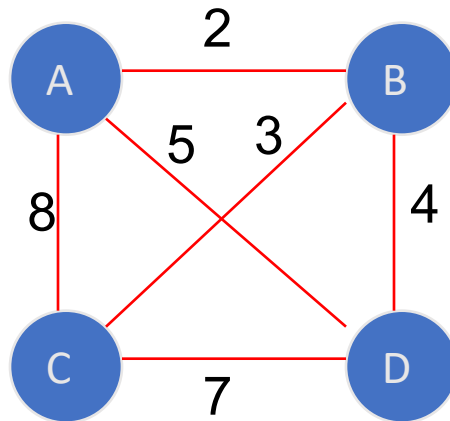
A brute force solution to a problem involving search for an element with a special property, usually among combinatorial objects such as permutations, combinations, or subsets of a set.

Method:

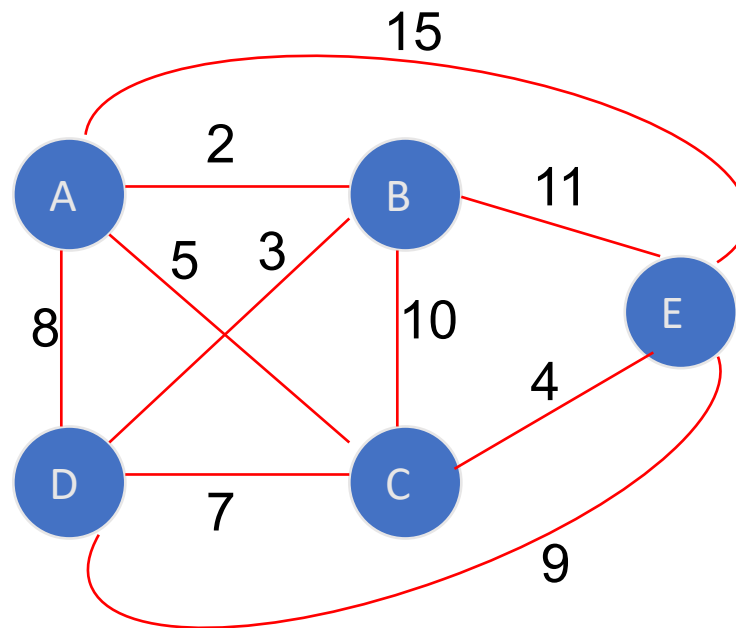
- generate a list of all potential solutions to the problem in a systematic manner (see algorithms in Sec. 5.4)
- evaluate potential solutions one by one, disqualifying infeasible ones and, for an optimization problem, keeping track of the best one found so far
- when search ends, announce the solution(s) found

Example 1: Traveling Salesman Problem

- Given n cities with known distances between each pair, find the shortest tour that passes through all the cities exactly once before returning to the starting city
- Alternatively: Find shortest *Hamiltonian circuit* in a weighted connected graph
- Example:



Example 2: Traveling Salesman Problem



TSP by Exhaustive Search

Tour	Cost
$a \rightarrow b \rightarrow c \rightarrow d \rightarrow a$	$2+3+7+5 = 17$
$a \rightarrow b \rightarrow d \rightarrow c \rightarrow a$	$2+4+7+8 = 21$
$a \rightarrow c \rightarrow b \rightarrow d \rightarrow a$	$8+3+4+5 = 20$
$a \rightarrow c \rightarrow d \rightarrow b \rightarrow a$	$8+7+4+2 = 21$
$a \rightarrow d \rightarrow b \rightarrow c \rightarrow a$	$5+4+3+8 = 20$
$a \rightarrow d \rightarrow c \rightarrow b \rightarrow a$	$5+7+3+2 = 17$

More tours?

Less tours?

Efficiency:

Solving Traveling Salesman Problem

- A Hamiltonian circuit is a sequence of **$n+1$** adjacent vertices $v_{i0}, v_{i1}, \dots, v_{in-1}, \dots v_{i0}$, where the 1st and last vertices are the same while all other **$n-1$** vertices are distinct.
- Thus the tours are generated by the **permutation** of **$n-1$** intermediate cities.

TSP

```
min=1000000;  
for b=2 to 4  
    distance=distance+d[a,b];  
    for c=2 to 4  
        if (b!=c)  
            for d=2 to 4  
                if(b!=d and c !=d)  
                    distance=distance+d[b,c]+d[c,d]  
            distance=distance+d[d,a]  
        if distance < min  
            min = distance
```

How's about n=5 or 6?

Example 2: Knapsack Problem

Given n items:

- weights: $w_1 \ w_2 \ \dots \ w_n$
- values: $v_1 \ v_2 \ \dots \ v_n$
- a knapsack of capacity W

Find most valuable subset of the items that fit into the knapsack

Example: Knapsack capacity $W=16$

<u>item</u>	<u>weight</u>	<u>value</u>
1	2	\$20
2	5	\$30
3	10	\$50
4	5	\$10

Knapsack Problem by Exhaustive Search

<u>Subset</u>	<u>Total weight</u>	<u>Total value</u>
{1}	2	\$20
{2}	5	\$30
{3}	10	\$50
{4}	5	\$10
{1,2}	7	\$50
{1,3}	12	\$70
{1,4}	7	\$30
{2,3}	15	\$80
{2,4}	10	\$40
{3,4}	15	\$60
{1,2,3}	17	not feasible
{1,2,4}	12	\$60
{1,3,4}	17	not feasible
{2,3,4}	20	not feasible
{1,2,3,4}	22	not feasible

<u>item</u>	<u>weight</u>	<u>value</u>
1	2	\$20
2	5	\$30
3	10	\$50
4	5	\$10

Efficiency: ?

$\Omega(2^n)$

NP-hard problem

No polynomial-time algorithm

Example 3: The Assignment Problem

There are n people who need to be assigned to n jobs, one person per job. The cost of assigning person i to job j is $C[i,j]$. Find an assignment that minimizes the total cost.

	Job 0	Job 1	Job 2	Job 3	
Person 0	9		2	7	8
Person 1	6	4	3	7	
Person 2	5	8	1	8	
Person 3	7	6	9	4	

Algorithmic Plan: Generate all legitimate assignments, compute their costs, and select the cheapest one.

How many assignments are there?

Pose the problem as the one about a cost matrix:

	Job 0	Job 1	Job 2	Job 3
Person 0	9	2	7	8
Person 1	6	4	3	7
Person 2	5	8	1	8
Person 3	7	6	9	4

	Job 0	Job 1	Job 2	Job 3
Person 0	9	2	7	8
Person 1	6	4	3	7
Person 2	5	8	1	8
Person 3	7	6	9	4

	Job 0	Job 1	Job 2	Job 3
Person 0	9	2	7	8
Person 1	6	4	3	7
Person 2	5	8	1	8
Person 3	7	6	9	4

Assignment Problem by Exhaustive Search

$$C = \begin{bmatrix} 9 & 2 & 7 & 8 \\ 6 & 4 & 3 & 7 \\ 5 & 8 & 1 & 8 \\ 7 & 6 & 9 & 4 \end{bmatrix}$$

Assignment (col.#s)

Total Cost

1, 2, 3, 4

$9+4+1+4=18$

1, 2, 4, 3

$9+4+8+9=30$

1, 3, 2, 4

$9+3+8+4=24$

1, 3, 4, 2

$9+3+8+6=26$

1, 4, 2, 3

$9+7+8+9=33$

1, 4, 3, 2

$9+7+1+6=23$

etc.

(For this particular instance, the optimal assignment can be found by exploiting the specific features of the number given. It is:)

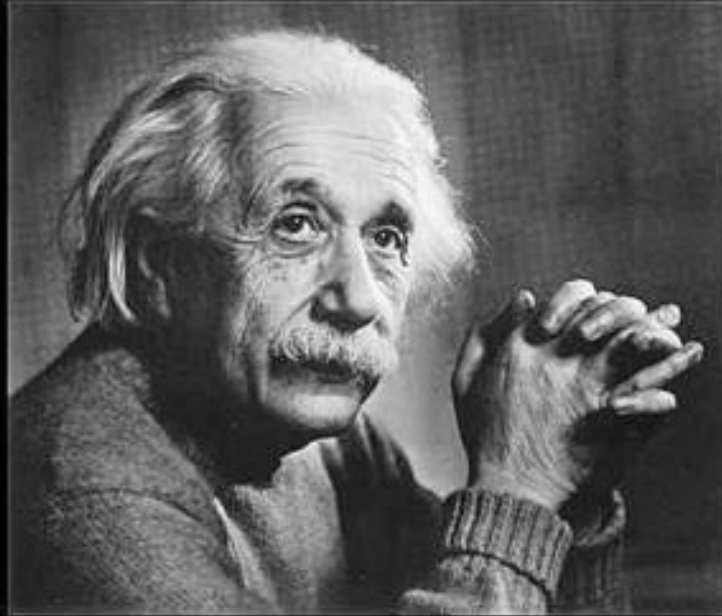
Final Comments on Exhaustive Search

- Exhaustive-search algorithms run in a realistic amount of time only on very small instances
- In some cases, there are much better alternatives!
 - Euler circuits
 - shortest paths
 - minimum spanning tree
 - assignment problem
- In many cases, exhaustive search or its variation is the only known way to get exact solution

Homework

- Finish 4x4 assignment problem by listing all situations.
- Write program of the assignment program whose size is 4x4.

Do not Underestimate Brute Force



"The world will not be destroyed by those who do evil, but by those who watch them without doing anything." - Albert Einstein